Qué problema estamos resolviendo (Adding Problem)

Entrada: una secuencia de longitud T. En cada instante tenemos **dos features**:

$$\mathbf{x}_t = [x_t, m_t]$$

- $x_t \in [0, 1]$: un número aleatorio.
- $m_t \in \{0, 1\}$: una "marca" que indica si ese paso temporal es relevante.

Regla: en cada secuencia hay exactamente **dos marcas** $m_t = 1$.
Salida:

$$y = \sum_{t=1}^{T} m_t \, x_t$$

Como solo hay dos marcas, esto equivale a:

$$y = x_{t_1} + x_{t_2}$$

donde $t_1$está en la primera mitad y $t_2$en la segunda.

Por qué es ideal para las redes recurrentes

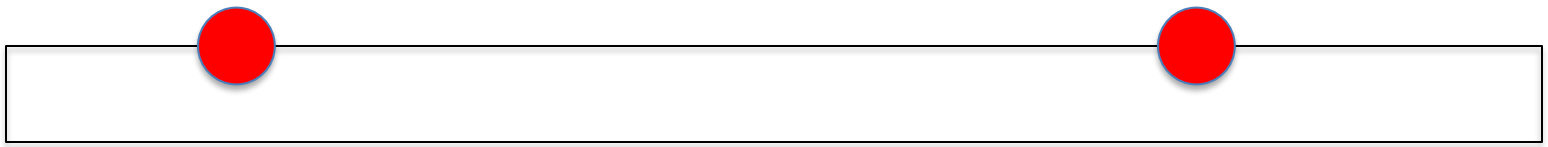
- el modelo debe "leer" un valor importante al principio (en t_1)
- **guardarlo** durante muchos pasos
- y luego sumarlo con el segundo valor (en t_2?

Esto fuerza **memoria a largo plazo** y muestra la ventaja de las puertas.

Why is this Task Difficult?

- Relevant positions can be far apart
  - The model must:
    - Detect a marked position.
    - Store the corresponding value.
    - Keep it in memory across many time steps.
    - Add it to the second marked value.
  - Exposes vanishing/exploding gradient issues

Visual Representation of the Adding Problem



The model must remember the first marked value until it encounters the second marked position.

Experimental Setup

- Sequence length: T=200
- Input dimension: 2 (value + marker)
- Output: regression (real number)
- Loss: Mean Squared Error (MSE)
- Optimizer: Adam

Architectures compared:

- SimpleRNN(32)
- GRU(32)
- LSTM(32)

Results

	Model	Validation MSE	Test MSE
T=200 n_samples = 60000	SimpleRNN	0.1629	0.1658
	GRU	0.0023	0.0023
	LSTM	0.0608	0.0597
T=100 n_samples = 10000	SimpleRNN	0.1676	0.1639
	GRU	0.1612	0.1579
	LSTM	0.1659	0.1632
T=1000 n_samples = 60000	SimpleRNN	0.1671	0.1653
	GRU	0.0010	0.0010
	LSTM	0.0519	0.0510

Results

n_samples=10000, T=100

Model	Test MSE
SimpleRNN	0.1639
GRU	0.1579
LSTM	0.1632

Secuencia más corta + pocos datos →
Todos fallan (≈ 0.16)

n_samples=60000, T=200

Model	Test MSE
SimpleRNN	0.1658
GRU	0.0023
LSTM	0.0597

Dependencia larga (T=200) + datos suficientes →
GRU ≫ LSTM > SimpleRNN

Model	Test MSE
SimpleRNN	0.1653
GRU	0.0010
LSTM	0.0510

Dependencia muy larga + muchos datos →
GRU domina claramente

Cuando T es pequeño o hay pocos datos:
→ Todos los modelos fallan o se comportan parecido

Cuando T es grande y hay suficientes datos:
→ SimpleRNN no puede mantener información estable.
→ GRU (y en menor medida LSTM) sí lo consigue.

Repaso - Qué significan val_mse y test_mse

MSE (Mean Squared Error)

- Es el error cuadrático medio:

$$\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2$$

  - Si el modelo se equivoca mucho, MSE será grande.
  - Si el modelo predice casi perfecto, MSE será muy pequeño.

val_mse

Es la MSE sobre el **conjunto de validación** al final del entrenamiento.

  - medir generalización mientras entrenas
  - detectar sobreajuste
  - comparar modelos sin tocar test todavía

test_mse

Es la MSE sobre el **conjunto de test**, que idealmente se usa solo al final como evaluación final.

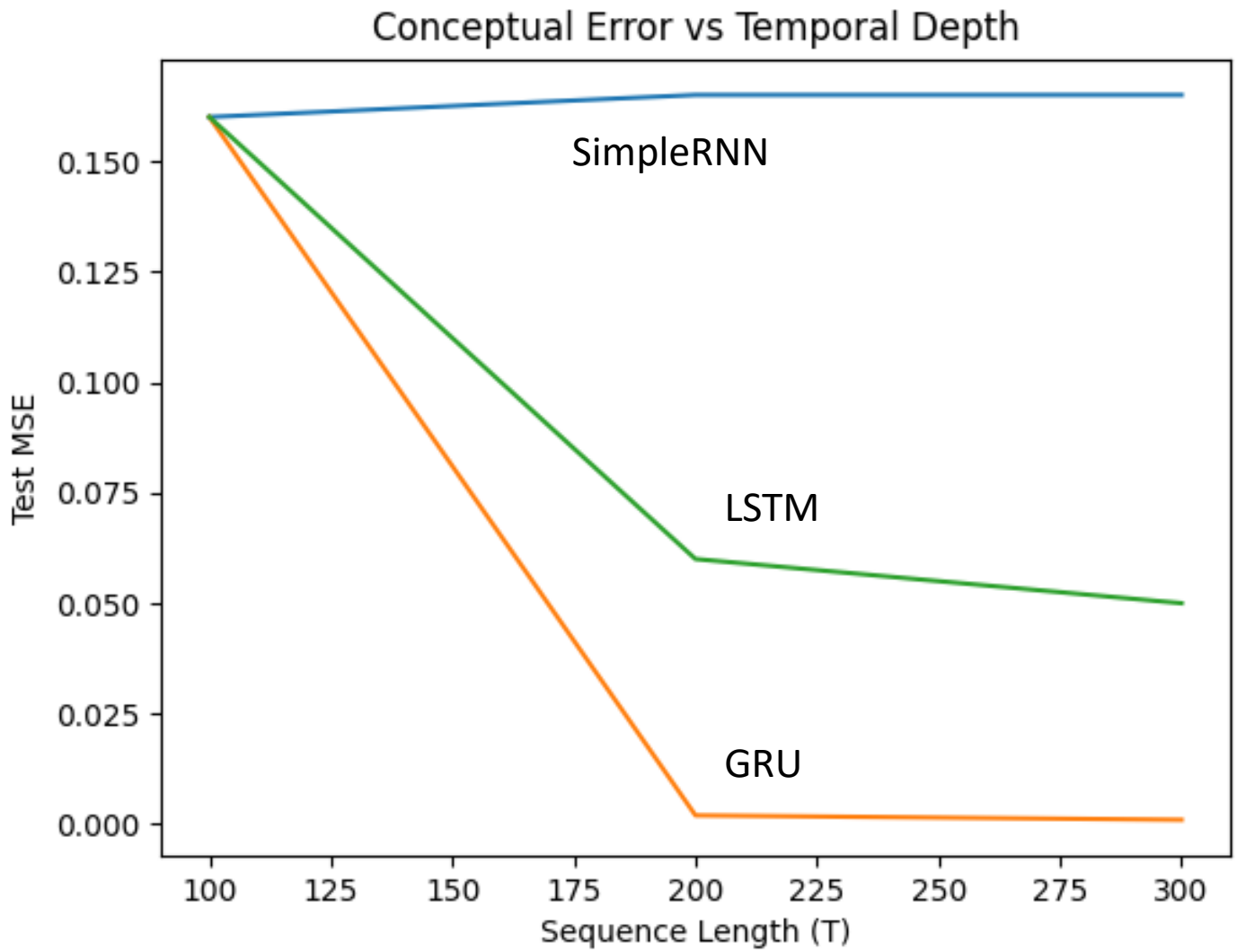
- Si val_mse ≈ test_mse, el modelo generaliza bien.
- Si val_mse << test_mse, puede haber sobreajuste o leakage.
- Si ambos son grandes, no aprende.

Interpretación de los resultados

- SimpleRNN: test_mse ≈ 0.1653
- GRU: test_mse ≈ 0.0010
- LSTM: test_mse ≈ 0.0510

- **GRU**: aprende casi perfecto (error muy pequeño).
- **LSTM**: mejora respecto a SimpleRNN, pero no tanto como GRU en este setup.
- **SimpleRNN**: no consigue mantener correctamente la información y acumula error.

Key Insight

- SimpleRNN applies repeated nonlinear transformations
  - Memory degrades over long sequences
  - GRU/LSTM introduce gated memory paths
  - Gating stabilizes gradient propagation

Error vs Temporal Depth